

Diseño de IOT y sistemas embebidos

Laboratorio 3

FreeRTOS

Docentes:

Nicolas Calarco y Pablo Alonso

Estudiantes:

Victoria Amor, Nicolas Meyer y Bruno Moreira

28 de Mayo de 2026

Introducción

En este laboratorio se desarrolló una arquitectura de software basada en multitasking utilizando FreeRTOS. A diferencia de los laboratorios anteriores, donde el programa seguía una ejecución más secuencial, en esta práctica se buscó separar el funcionamiento del sistema en distintas tareas concurrentes, cada una con una función específica. Se hace uso de tareas, colas, semáforos, timers de software y memoria dinámica con el objetivo de controlar un LED RGB a partir de comandos recibidos por UART.

El sistema implementado permite ingresar comandos de texto desde el monitor, indicando el color deseado y el tiempo de espera antes de aplicarlo. Estos comandos son recibidos por una tarea encargada de la comunicación UART, luego se envían a una cola y finalmente son procesados por otra tarea, que crea un timer para ejecutar el cambio de color en el momento correspondiente. Mientras tanto, una tercera tarea mantiene el parpadeo del LED utilizando el color actual del sistema.

Para evitar problemas al acceder a datos compartidos, el color actual del LED se protege mediante un mutex. Además, se agregaron funcionalidades como la consulta del estado del sistema, la cancelación de timers pendientes y el monitoreo del stack.

1. Arquitectura del sistema

La estructura del proyecto se organizó dentro de la carpeta `components/` donde se creó un componente para `task_a`, encargado del parpadeo del LED RGB; un componente para `task_b`, encargado de la recepción de comandos por UART; un componente para `task_c`, encargado de la creación y administración de timers y un componente compartido llamado `shared_types`, donde se definieron los tipos de datos y las variables globales utilizadas por más de una tarea. Además, se mantuvo el componente correspondiente al driver del LED RGB, reutilizando la lógica necesaria para controlar el LED WS2812.

En el archivo principal `main.c`, se realiza la inicialización general del sistema. Primero se define un color inicial para el LED (rojo) y se crea el mutex que protege el acceso a dicho color. Si la inicialización falla, el sistema registra el error y no continúa con la creación de tareas. Luego se inicializa el LED RGB.

Posteriormente se crea una cola destinada a transportar comandos entre tareas. Esta cola almacena elementos de tipo `led_command_t`, una estructura que contiene dos datos: el color que se desea aplicar y el tiempo de espera antes de aplicarlo. La cola actúa como un intermediario entre la tarea que recibe comandos y la tarea que los procesa.

Después de inicializar los recursos compartidos, se crean las tres tareas principales mediante `xTaskCreate()`. Esta función permite registrar una nueva tarea en FreeRTOS, indicando qué función ejecutará, qué tamaño de stack tendrá, qué parámetro recibirá y con qué prioridad se ejecutará. Task A no necesita recibir parámetros, porque no trabaja directamente con el handle del LED. En su lugar, utiliza las funciones de la librería `rgb_led`, que ya mantiene internamente el puntero al LED inicializado. Por eso, al crear task A, se pasa NULL como parámetro.

En cambio, task B y task C sí reciben parámetros, porque ambas necesitan acceder a la cola de comandos. Para eso se crean estructuras de parámetros donde se guarda el handle de la cola. Task b usa ese handle para enviar comandos con `xQueueSend()`, mientras que task c lo utiliza para recibirlos con `xQueueReceive()`.

Las prioridades se asignaron de forma relativa a partir de `tskIDLE_PRIORITY` como referencia. Esta constante representa la prioridad de la tarea *Idle* de FreeRTOS, es decir, la tarea de menor prioridad del sistema que se ejecuta cuando no hay otras tareas listas para correr y por default se encarga de liberar los recursos asociados a tareas que fueron eliminadas. A partir de esta referencia, task a se configuró con la prioridad más baja del sistema implementado, es decir, `tskIDLE_PRIORITY + 1`, ya que su funcionamiento es periódico y no requiere una respuesta inmediata. Task c se configuró con una prioridad intermedia, `tskIDLE_PRIORITY + 2`. Finalmente, task B se configuró con la mayor prioridad dentro de las tareas creadas, `tskIDLE_PRIORITY + 3`, debido a que debe responder con más rapidez a la llegada de caracteres desde la computadora.

Antes de continuar se hacen algunas aclaraciones. Para facilitar la depuración y verificar el flujo de ejecución del programa, se incorporó logging en los distintos archivos mediante las macros de ESP-IDF. En cada módulo se definió un identificador propio mediante una variable TAG, lo que permite reconocer desde qué tarea o archivo proviene cada mensaje mostrado en el monitor.

Además, se utilizaron distintos niveles de logging según la importancia del evento registrado. El macro `ESP_LOGI()` se utilizó para informar eventos normales del sistema, como el inicio de una tarea, la creación de la cola, la recepción de un comando válido, la creación de un timer o la aplicación de un color. El macro `ESP_LOGW()` se utilizó para advertencias no fatales, por ejemplo cuando se recibe un comando inválido, cuando la cola está llena o cuando no se puede tomar un mutex en una situación determinada. Finalmente, `ESP_LOGE()` se reservó para errores más importantes, como fallas al crear el mutex, la cola, una

tarea o un timer. De esta forma, el monitor serie permite seguir el comportamiento del sistema y detectar en qué etapa ocurre un problema.

La creación de tareas se realizó mediante `xTaskCreate()`, que devuelve un valor de tipo `BaseType_t`. Este valor permite saber si la tarea fue creada correctamente. Cuando la función devuelve `pdPASS`, la tarea fue registrada y queda pronta para ser planificada por el scheduler. En cambio, si no se obtiene `pdPASS`, significa que ocurrió un error durante la creación, por ejemplo por falta de memoria disponible para reservar el stack de la tarea. Por ese motivo, luego de cada llamada a `xTaskCreate()` se verifica el valor devuelto y, si la tarea no pudo crearse, se registra el error mediante `ESP_LOGE()` y se detiene la inicialización.

En este caso, cada tarea se creó con un tamaño de stack de 4096. Este valor indica la cantidad de memoria reservada para la pila de la tarea, donde se almacenan variables locales, direcciones de retorno y datos temporales usados durante la ejecución. Se eligió un valor suficientemente grande para evitar desbordamientos de stack durante las pruebas, especialmente porque algunas tareas realizan procesamiento de texto, logging y creación de estructuras auxiliares. Posteriormente, como mejora, se incorporó el uso de `uxTaskGetStackHighWaterMark()`, que permite observar cuánta memoria de stack queda disponible como mínimo durante la ejecución. Esto sirve para verificar si el tamaño elegido es adecuado o si podría reducirse en una versión más optimizada manualmente.

En muchos casos se guarda el handle de la tarea creada para poder referenciarla posteriormente, por ejemplo para eliminarla, suspenderla o modificarla desde otra tarea, en esta implementación no fue necesario, ya que las tres tareas principales deben permanecer ejecutándose durante todo el funcionamiento del sistema y no se modifican en otras tareas. Por eso, en la llamada a `xTaskCreate()` se pasó `NULL` en el último argumento. Esto indica que no se necesita guardar el identificador de la tarea creada.

2.Tarea A

Task `a` es la tarea encargada de controlar el parpadeo visible en el led. Su función no es decidir qué color debe mostrarse, sino leer el color actual del sistema y mostrarlo en el LED de forma periódica. Por eso, esta tarea se mantiene ejecutándose continuamente dentro de un bucle infinito.

Recordar que el cuarto argumento (parámetro) se pasa como `NULL` porque la tarea no necesita recibir datos adicionales desde `main.c`. aunque la función

`task_a()` debe tener un parámetro de tipo `void *` por el formato exigido por FreeRTOS. Por eso, dentro de la función se pone `(void)pvParameters`, únicamente para indicar que el parámetro fue omitido de forma intencional y para evitar warnings del compilador.

Dentro del ciclo principal de la tarea, primero se declara una variable local de tipo `rgb_color_t`, llamada `local_color`. Esta variable se usa para guardar una copia del color actual. Para leer el color global `g_current_color`, la tarea toma previamente el mutex `g_color_mutex` mediante `xSemaphoreTake()`. Esto es necesario porque el color global puede ser modificado por otra parte del programa, especialmente por la tarea de transición que aplica los cambios de color. Una vez copiado el valor en `local_color`, el mutex se libera inmediatamente con `xSemaphoreGive()`.

Este procedimiento evita mantener el mutex tomado durante más tiempo del necesario. La tarea no enciende el LED mientras mantiene el mutex bloqueado, sino que solo lo usa para copiar el dato compartido. Después trabaja con la copia local.

Una vez obtenido el color, task a llama a `rgb_led_set_color(local_color.r, local_color.g, local_color.b)`. Esta función pertenece a la librería `rgb_led` y se encarga de enviar al LED las componentes rojo, verde y azul correspondientes. Luego la tarea espera 500 ms mediante `vTaskDelay(pdMS_TO_TICKS(BLINK_ON_TIME_MS))`. El macro `pdMS_TO_TICKS()` convierte el tiempo expresado en milisegundos a ticks del sistema operativo, que es la unidad temporal utilizada por FreeRTOS según lo visto en clase.

Pasados los 500 ms, la tarea llama a `rgb_led_off()`, apagando el LED. Luego vuelve a esperar otros 500 ms. De esta forma, el LED queda encendido medio segundo y apagado medio segundo, generando un parpadeo constante.

3.Tarea B

Esta tarea se encarga de realizar las configuraciones pertinentes para el correcto funcionamiento del protocolo UART [1], se extrae la estructura principal de comunicación del ejemplo: [peripherals/uart/uart_echo](#).

Para setear y elegir correctamente los parámetros a configurar se utilizó la estructura de configuración “`uart_config_t`” en conjunto con la documentación de la placa ESP32-S2-Kaluga-1 [2].

Los parámetros seleccionados fueron:

ECHO_UART_PORT_NUM	115200
ECHO_UART_BAUD_RATE	UART_DATA_8_BITS
ECHO_TASK_STACK_SIZE	UART_PARITY_DISABLE
BUF_SIZE	UART_STOP_BITS_1
ECHO_TEST_TXD	UART_HW_FLOWCTRL_DISABLE
.source_clk	UART_SCLK_DEFAULT

Figura 1: Configuración protocolo uart.

```
ESP_ERROR_CHECK(uart_driver_install(ECHO_UART_PORT_NUM, BUF_SIZE * 2, 0, 0, NULL, 0));  
ESP_ERROR_CHECK(uart_param_config(ECHO_UART_PORT_NUM, &uart_config));  
ESP_ERROR_CHECK(uart_set_pin(ECHO_UART_PORT_NUM, ECHO_TEST_TXD, ECHO_TEST_RXD,  
UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));
```

Figura 2: Iniciación protocolo uart.

La figura 2 muestra el código utilizado para inicializar de forma segura, el driver, los parámetros lógicos y pines a utilizar por el UART, en este caso:

`uart_driver_install`: Avisa al sistema operativo (FreeRTOS) que vas a usar un periférico UART y le pide que reserve memoria RAM para manejarla.

`uart_param_config`: Va directamente a los registros físicos dentro del procesador y carga las configuraciones: la velocidad de transmisión baudios (bits por segundo), el tamaño de los paquetes y si funciona con paridad.

`uart_set_pin`: Enruta las señales internas hacia los pines físicos de cobre, en este caso 43 y 44.

A su vez se definieron estructuras auxiliares como `line_buffer`, `led_cmd` y `line_index` las cuales se utilizaron para almacenar la entrada del monitor y para el funcionamiento interno de la task.

Para el manejo del timing de la tarea se utilizó la función `uart_read_bytes` la cual, además de leer los datos de entrada, si no hay nada en el puerto UART, pausa la tarea durante 10ms. De esta forma, el bucle es infinito, pero no es bloqueante; cede el tiempo al resto del sistema de forma eficiente.

También se implementó `uart_write_bytes` para enviar los datos recibidos por el UART a la computadora mediante el TX, espejo en la consola.

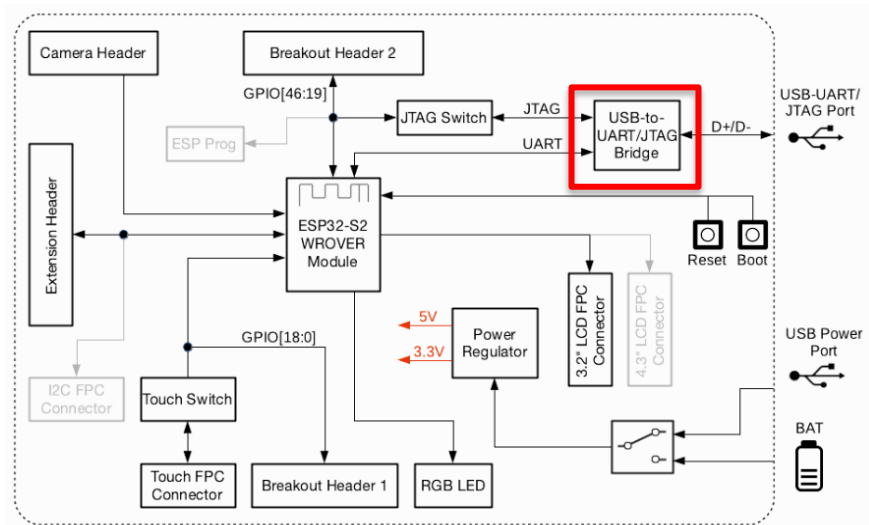


Figura 3: ESP32-S2-Kaluga-1 block diagram

Es importante resaltar que la placa se está comunicando mediante UART a un puerto USB los cuales son dos idiomas eléctricos y lógicos completamente diferentes. Es necesario un traductor, para el caso de la Kaluga-1 se utiliza un adaptador FT2232, representado en la figura 3 con un cuadrado rojo.

Como núcleo principal de la task, se implementó una lógica la cual permite evaluar los bytes recibidos mediante el UART para detectar cuando el usuario ingreso la tecla ENTER en la terminal, para posteriormente evaluar y dividir la estructura. Por último, se envía la estructura mediante el comando `xQueueSend()`.

Para conseguir este funcionamiento se implementaron bloques condicionales `if` los cuales se encargan de evaluar que se cumpla cada condición. Cuando detecta el final (`\n` o `\r`) prepara la estructura auxiliar usando `memset()`, el cual limpia el espacio de memoria a utilizar.

Por último, se utiliza una variable tipo `bool` `espacio_idx` y la función `atoi()` (la cual devuelve 0 si no es un numero) en conjunto con la evaluación de que el color sea válido para evaluar si el formato es el correcto. En el caso de serlo se carga a la estructura solicitada "led_command_t" para enviarla a `led_queue`.

4. Task C

La `task_c` será la encargada de tomar comandos desde la cola y crear timers *one-shot*, o de un solo uso, para ejecutar dichos comandos luego del tiempo indicado. Para esto, se utilizan las variables globales `led_queue`, que

corresponde al *handler* de la cola por donde llegan los comandos para la LED, `g_color_mutex`, que corresponde al *handler* del mutex que protege el valor RGB actual, y `g_current_color`, que guarda el color actual de la LED.

Luego se crea una función estática llamada `timer_callback()`, la cual se ejecuta cuando vence el timer *one-shot* creado por la Task C. Esta función recupera inicialmente el puntero que apunta al color de la LED, el cual fue guardado en `pvTimerID`, utilizando `pvTimerGetTimerID()`. Luego toma el mutex que protege el color actual de la LED, actualiza `g_current_color` con el nuevo valor RGB y libera el mutex para que pueda ser utilizado por otra tarea. Por último, libera la memoria dinámica que se utilizó para guardar el color mediante `vPortFree()`, y elimina el timer con `xTimerDelete()` para evitar que siga ocupando recursos.

La Task C en sí consiste en esperar que le llegue un comando desde la cola. Para esto, primero define una variable `led_command_t command`, haciendo referencia a la estructura definida en `shared_types.h`. Esta variable representa el comando recibido, el cual contiene el color que se desea aplicar a la LED y el tiempo de espera antes de aplicarlo.

Luego, la tarea queda bloqueada esperando que llegue un comando desde la cola mediante `xQueueReceive()`. Cuando recibe un comando, asigna memoria dinámica para manejar el color de la LED usando `pvPortMalloc(sizeof(rgb_color_t))`. Después copia el color recibido con `*color = command.color` y calcula el período del timer a partir del delay indicado en el comando, usando `TickType_t period = command.delay_s * 1000 / portTICK_PERIOD_MS`.

A continuación, se crea un timer *one-shot* llamado "led_timer" utilizando `xTimerCreate()`. Este timer se configura con el período calculado previamente, con el puntero al color como `pvTimerID`, y con la función `timer_callback()` como función que se ejecutará cuando el timer finalice. Finalmente, el timer creado se inicia mediante `xTimerStart()`, quedando pendiente hasta que se cumpla el tiempo indicado y se aplique el nuevo color a la LED.

5. Implementación de opcionales

5.1 Cola con timeout

La cola con timeout se implementó para evitar que task B quede bloqueada indefinidamente cuando intenta enviar un comando a la cola. Como esta tarea es la encargada de recibir los comandos por UART, no conviene que quede

detenida para siempre esperando espacio en la cola, porque eso podría impedir que siga atendiendo nuevos datos recibidos.

Para lograrlo, se modificó la llamada a `xQueueSend()`, reemplazando la espera indefinida por un tiempo máximo de espera. De esta forma, cuando task B recibe un comando válido, intenta enviarlo a la cola durante un intervalo limitado. Si en ese tiempo hay espacio disponible, el comando se encola correctamente y luego puede ser procesado por task C.

Si la cola permanece llena durante todo el tiempo de espera, el envío falla y `xQueueSend()` devuelve un valor distinto de `pdPASS`. En ese caso, el comando no queda guardado en la cola y se registra una advertencia con `ESP_LOGW()`. Esto permite detectar que el sistema recibió más comandos de los que pudo procesar en ese momento.

5.2 Cancelación de timers pendientes

La cancelación de timers pendientes se implementó dentro de task c, porque esta tarea es la encargada de recibir los comandos desde la cola y crear los timers que aplicarán los colores más adelante. El problema que se buscó resolver fue que no quedaran varios timers pendientes para el mismo color. Por ejemplo, si se enviaba primero ROJO 10 y luego ROJO 4, no tenía sentido conservar los dos timers, porque el segundo comando reemplaza al anterior para ese mismo color.

Para poder controlar esto, se creó una estructura auxiliar llamada `pending_timer_t`. Esta estructura guarda la información de cada timer pendiente: si está activo, qué color tiene asociado, el handle del timer y el puntero a la memoria dinámica donde se guardó ese color. Luego se declaró un arreglo de estas estructuras llamado `pending_timers`, con el mismo largo que la cola. De esta forma, el programa puede llevar un registro de los timers que fueron creados pero que todavía no vencieron.

El array `pending_timers` funciona como una tabla de timers activos. Cada posición del arreglo puede estar libre u ocupada. Para saberlo, se utiliza el campo `active`. Por eso, cuando se crea un timer nuevo, se llama a `register_pending_timer()`, que recorre el arreglo desde la primera posición hasta encontrar una entrada libre. En esa posición guarda el color, el handle del timer y el puntero `color_ptr`. Además, marca la entrada como activa y aumenta el contador global `g_pending_timers`.

Cuando task C recibe un nuevo comando desde la cola, antes de crear el nuevo timer llama a la función `cancel_previous_timer_same_color`

(`command.color`). Esa función recorre el arreglo `pending_timers` buscando si ya existe un timer activo cuyo color sea igual al color del nuevo comando. Para comparar los colores se usa una función auxiliar, `same_color()`, que compara las tres componentes RGB: rojo, verde y azul. Solo si las tres componentes coinciden se considera que es el mismo color, devolviendo `true`.

Si se encuentra un timer activo del mismo color, se cancela solamente ese timer. Primero se llama a `xTimerStop()` para detenerlo y después a `xTimerDelete()` para eliminarlo. Luego se libera con `vPortFree()` la memoria dinámica que estaba asociada a ese timer, porque al cancelarlo su callback ya no se va a ejecutar y, por lo tanto, nadie más liberaría esa memoria. Después se limpia la entrada correspondiente del arreglo, dejando active en `false`, el timer en `NULL` y el puntero de color también en `NULL`. También se disminuye `g_pending_timers`.

La función hace `return` apenas cancela un timer. Esto es importante porque la idea no fue borrar todos los timers del sistema, sino cancelar únicamente el timer previo del mismo color.

5.3 Comando de estado

El comando `STATUS` se implementó para poder consultar el estado actual del sistema desde la terminal `UART`. A diferencia de los comandos de color, este comando no genera un nuevo cambio en el led ni crea un timer, sino que funciona como una consulta de diagnóstico. La consigna propone que task B responda por `UART` con el color actual y con la cantidad de timers pendientes.

La estrategia utilizada consistió en detectar la palabra `STATUS` durante el parseo de los comandos recibidos por `UART`. Cuando task B recibe este comando, no lo encola hacia task C, porque no hay ningún color nuevo que aplicar ni ningún retardo que programar. En lugar de eso, la propia task B consulta el estado actual del sistema y envía una respuesta directamente por `UART`.

Para obtener el color actual, se accede a la variable global compartida `g_current_color`. Como esa variable también puede ser modificada por otras partes del programa, se protege el acceso mediante el mutex `g_color_mutex`. Entonces, task B toma el mutex, copia el valor actual del color a una variable local y libera el mutex inmediatamente.

Además, se mantiene un contador de timers pendientes. Este contador aumenta cuando se crea un nuevo timer y disminuye cuando un timer vence o se cancela. De esa forma, al recibir `STATUS`, el sistema puede informar cuántos cambios de color están programados pero todavía no fueron aplicados.

5.4 Monitor de stack

Uno de los opcionales implementados fue el monitoreo del stack de las tareas mediante la función `uxTaskGetStackHighWaterMark()`. Esta función permite consultar cuánta memoria de stack quedó libre en el peor caso desde que la tarea comenzó a ejecutarse. De esta forma, no se observa simplemente el stack libre en un instante puntual, sino el mínimo stack disponible durante toda la ejecución previa de la tarea.

Para esto, cada tarea utiliza un contador interno de ciclos de ejecución, denominado `cycle_counter`, que permite no consultar el stack en cada vuelta del bucle, sino hacerlo únicamente cada cierta cantidad de ciclos. Esto evita llenar la consola con mensajes repetidos y permite observar el uso de memoria de forma más ordenada durante la ejecución.

En cada tarea, el contador se incrementa dentro del ciclo principal. Cuando alcanza el valor definido en el código, se reinicia y se llama a la función `uxTaskGetStackHighWaterMark(NULL)`. Esta función devuelve el mínimo stack libre que tuvo la tarea desde que comenzó a ejecutarse, por lo que permite estimar si el tamaño de stack asignado en `xTaskCreate()` fue suficiente.

El parámetro `NULL` indica que la consulta se realiza sobre la tarea que está ejecutando la función en ese momento. El valor obtenido se muestra mediante `ESP_LOGI()`, permitiendo observar en el monitor si el tamaño de stack asignado al crear la tarea es adecuado. Si el valor reportado fuera muy bajo, significaría que la tarea estuvo cerca de agotar su stack y sería necesario aumentar el tercer argumento de `xTaskCreate()`. En cambio, si el valor fuera muy alto, podría reducirse el tamaño asignado para optimizar el uso de memoria. En este caso, el monitoreo no modifica automáticamente el stack de la tarea. Su función es únicamente informar el mínimo stack libre observado, para que el ajuste del valor de `usStackDepth` pueda realizarse manualmente en el código.

5. Conclusión

Se puede concluir que se logró la implementación un sistema de control temporizado de un LED mediante los comandos recibidos por la terminal en una placa de desarrollo ESP32-S2 Kaluga1. Integrando para su funcionamiento conceptos básicos de RTOS, como colas, semáforos, timers, stacks y el manejo de tareas como su intercomunicación.

Este proyecto se desarrollo en equipo por lo cual se utilizo un repositorio de GitHub: https://github.com/IOT-EAMTA-2026/Laboratorio_3_FreeRTOS.

Referencias

- [1] Espressif Systems, "Universal Asynchronous Receiver/Transmitter (UART) - ESP-IDF Programming Guide," Espressif Documentation. [En línea]. Disponible en: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html>. [Accedido: 17-may-2026].
- [2] Espressif Systems, "ESP32-S2-Kaluga-1 Kit v1.2 - ESP-IDF Programming Guide," Espressif Documentation. [En línea]. Disponible en: <https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32s2/hw-reference/esp32s2/user-guide-esp32-s2-kaluga-1-kit-v1.2.html#hardware-reference>. [Accedido: 23-may-2026].